Veri-Store Security Model

Mac Payton & Albert Huynh

March 15, 2026
Version 0.1.1

Contents

1 Introduction **6**

1.1 Document Purpose 6

1.2 System Overview 6

2 Scoping **7**

2.1 In Scope 7

2.2 Out of Scope 7

3 Use Scenarios **8**

3.1 Primary Use Case 8

3.2 Anti-Scenarios 8

4 Dependencies **9**

4.1 External Libraries 9

4.2 System Dependencies 9

5 Implementation Assumptions **10**

6 Trust Levels **11**

6.1 Administrator 11

6.2 Authenticated Client . . . 11
6.3 Storage Server . . . 11
6.4 Untrusted Network . . . 11

7 Entry Points **13**
7.1 Client API . . . 13
7.2 Inter-Server Communication . . . 13
7.3 Storage Subsystem . . . 13

8 Protected Assets **14**
8.1 Data Blocks . . . 14
8.2 Erasure-Coded Fragments . . . 14
8.3 Homomorphic Fingerprints . . . 14
8.4 System Resources . . . 14

9 Data Flow Diagram **15**
9.1 DFD Component Descriptions . . . 16
9.1.1 Client . . . 16
9.1.2 Encode Block . . . 16
9.1.3 Compute Fingerprint . . . 16
9.1.4 Distribute Fragments . . . 16
9.1.5 Network . . . 16
9.1.6 Fragment Storage . . . 16
9.1.7 Verify Fragments . . . 16
9.1.8 Reconstruct Block . . . 16

10 Threat Analysis: STRIDE **18**
10.1 Spoofing . . . 18
10.1.1 S.1: Client Impersonation . . . 18
10.1.2 S.2: Server Impersonation . . . 18

10.1.3 S.3: Fingerprint Source Forgery . . . 18
10.2 Tampering . . . 18
10.2.1 T.1: Fragment Corruption in Storage . . . 18
10.2.2 T.2: Fragment Modification in Transit . . . 18
10.2.3 T.3: Fingerprint Tampering . . . 18
10.2.4 T.4: Erasure Code Parameter Manipulation . . . 18
10.3 Repudiation . . . 18
10.3.1 R.1: Anonymous Data Storage . . . 18
10.3.2 R.2: Fragment Deletion Without Logging . . . 18
10.3.3 R.3: Untrackable Verification Failures . . . 18
10.4 Information Disclosure . . . 18
10.4.1 I.1: Fragment Eavesdropping . . . 18
10.4.2 I.2: Storage Server Data Access . . . 18
10.4.3 I.3: Fingerprint Analysis . . . 18
10.4.4 I.4: Error Message Information Leakage . . . 18
10.5 Denial of Service . . . 18
10.5.1 D.1: Storage Exhaustion . . . 18
10.5.2 D.2: Fragment Unavailability . . . 18
10.5.3 D.3: Verification Flooding . . . 18
10.5.4 D.4: Network Bandwidth Exhaustion . . . 18
10.6 Elevation of Privilege . . . 18
10.6.1 E.1: Code Injection via Malformed Input . . . 18
10.6.2 E.2: Verification Bypass . . . 18
10.6.3 E.3: Cryptographic Parameter Override . . . 18
10.6.4 E.4: Server-to-Server Privilege Escalation . . . 18

11 Threat Resolutions **19**
11.1 Spoofing Threat Resolutions . . . 19
11.1.1 S.1: Client Impersonation . . . 19

11.1.2 S.2: Server Impersonation . . . 19
11.1.3 S.3: Fingerprint Source Forgery . . . 19
11.2 Tampering Threat Resolutions . . . 19
11.2.1 T.1: Fragment Corruption in Storage . . . 19
11.2.2 T.2: Fragment Modification in Transit . . . 20
11.2.3 T.3: Fingerprint Tampering . . . 20
11.2.4 T.4: Erasure Code Parameter Manipulation . . . 20
11.3 Repudiation Threat Resolutions . . . 20
11.3.1 R.1: Anonymous Data Storage . . . 20
11.3.2 R.2: Fragment Deletion Without Logging . . . 20
11.3.3 R.3: Untrackable Verification Failures . . . 21
11.4 Information Disclosure Threat Resolutions . . . 21
11.4.1 I.1: Fragment Eavesdropping . . . 21
11.4.2 I.2: Storage Server Data Access . . . 21
11.4.3 I.3: Fingerprint Analysis . . . 21
11.4.4 I.4: Error Message Information Leakage . . . 21
11.5 Denial of Service Threat Resolutions . . . 22
11.5.1 D.1: Storage Exhaustion . . . 22
11.5.2 D.2: Fragment Unavailability . . . 22
11.5.3 D.3: Verification Flooding . . . 22
11.5.4 D.4: Network Bandwidth Exhaustion . . . 22
11.6 Elevation of Privilege Threat Resolutions . . . 22
11.6.1 E.1: Code Injection via Malformed Input . . . 22
11.6.2 E.2: Verification Bypass . . . 23
11.6.3 E.3: Cryptographic Parameter Override . . . 23
11.6.4 E.4: Server-to-Server Privilege Escalation . . . 23

12 External Security Notes **24**

12.1 Deployment Recommendations . . . 24

12.2 Known Limitations 24

12.3 Integration Considerations 24

13 Verification and Testing Plan **25**

13.1 Security Test Suite 25

13.2 Code Review Process 25

13.3 Regression Testing 25

14 Document Maintenance **26**

14.1 Review Schedule 26

14.2 Change Process 26

14.3 Version History 26

1 Introduction

1.1 Document Purpose

This document will be used to identify threats, design mitigations, and verify the security properties of the Veri-Store system throughout the development lifecycle. It will be updated as components are implemented and new threats are discovered.

1.2 System Overview

Veri-Store is a distributed file storage system built using 3-of-5 Reed-Solomon erasure coding. It splits a single data block into 5 fragments in such a way that the original block of data can be built with only 3 of the original fragments.

The main security concerns are (a) ensuring integrity of the fragments and (b) ensuring that a fragment belongs to the data block being reconstructed. If a fragment has been tampered with (integrity) or was not part of the original block (authenticity), but data block construction goes forward anyway, an adversary could inject malicious code into the system.

Veri-Store addresses these concerns using homomorphic fingerprinting, allowing each fragment to be independently verified on their respective servers as belonging to the correct data block without needing to reconstruct the full block.

2 Scoping

2.1 In Scope

2.2 Out of Scope

3 Use Scenarios

We expect a number of possible use cases of the Veri-Store system, given in the non-exhaustive list below:

- Intranet for an organization, providing remote access to documents and files to organization members.
- Academic research and testing for erasure coding systems.
- Static asset distribution across geographically dispersed servers.
- Medical records storage, providing both fault tolerance and tamper detection needed for legal compliance.
- Archival storage for organizations that need redundancy without the cost of full replication.

3.1 Primary Use Case

Users will access Veri-Store through a client application, using it to store files which can be accessed later from the network. The client application can store data, retrieve files, verify integrity, and reconstruct the original data block. We will start with a single-user application (akin to a flash drive over the network), with hopes to move Veri-Store to a multi-user system where files can be shared.

The servers in this use case are to be semi-trusted - we assume that adversaries will attempt to tamper with data fragments if they are able to break into the system. (See Section 6 for a formal discussion of trust levels.) Although a network environment (both LAN and public internet) is the goal, for the sake of this project, we will be hosting each "server" on a single machine through different `localhost` ports.

3.2 Anti-Scenarios

The following use cases are unsupported and/or known to be insecure:

- Running servers on machines which are wholly untrusted or known to be compromised.
- Applications with strict latency requirements. Erasure coding and fragment verification introduce non-trivial computational overhead.
- Unencrypted sensitive data - Veri-Store is not an encryption scheme. Sensitive data should be encrypted before being given to Veri-Store.
- We cannot guarantee availability if more than 2 of the 5 servers are unavailable.

4 Dependencies

4.1 External Libraries

Mathematical operations and erasure coding

- `galois` [$\geq$ `v0.3.8`]: Finite field arithmetic over $\mathbb{F}_{256}$, primarily utilized in developing Cauchy matrices
- `numpy` [$\geq$ `v1.26.0`]: Array operations and polynomial manipulation
- `reedsolo` [$\geq$ `v1.7.0`] (*optional*): Encoding and decoding data before dispersal

Web framework

- `fastapi` [$\geq$ `v0.110.0`]: REST API for client-server communication
- `uvicorn` [$\geq$ `v0.29.0`]: ASGI server for hosting the API

HTTP client

- `httpx` [$\geq$ `v0.27.0`]: For inter-server communication and fragment distribution

API data validation

- `pydantic` [$\geq$ `v2.6.0`]: Data validation and parsing for API requests/responses

Testing

- `pytest` [$\geq$ `v8.0.0`]: For running unit and integration tests
  - `pytest-asyncio` [$\geq$ `v0.23.0`]: For testing asynchronous code paths

4.2 System Dependencies

- **Network stack**:
- **File system**:
- **Operating system**:

5 Implementation Assumptions

- At most f out of n total servers may be compromised or fail and the system will still maintain data integrity
- Network communication channels are/will be authenticated
- Storage servers have sufficient disk space for fragments of size $1/m$ of original block
- Homomorphic fingerprint collision probability is negligible
- Erasure coding library correctly implements Reed-Solomon

6 Trust Levels

6.1 Administrator

System administrators have full access to the servers hosting the veri-store system and the stored data. They are responsible for maintaining the infrastructure, applying updates, and ensuring the system is running smoothly. The security model assumes that administrators are trusted and will not intentionally compromise data integrity or confidentiality. However, they have the capability to access stored fragments and fingerprints, so it is important to consider this trust level when designing mitigations for information disclosure threats.

Example System Administrators:

- Developers (e.g., Albert and Mac) during development and testing
- IT staff responsible for server/data maintenance in a production deployment
- Cloud provider administrators if deployed on cloud infrastructure (e.g., AWS, Azure)
- System operators with root access to physical or virtual servers
- *Additional examples to be added*

6.2 Authenticated Client

The term "authenticated client" refers both to the end users and the application they use to access the Veri-Store system.

An authenticated client has verified their identity to the system and is authorized to store and retrieve their own data. They are assumed to be legitimate, but may make mistakes or run buggy software which can violate system security. They should not have access to other users' data or have the ability to violate system integrity.

6.3 Storage Server

6.4 Untrusted Network

We assume a TLS connection between clients and servers, although we do not rely on it for data integrity. Adversaries will be able to observe network traffic, and it is assumed that they will, in theory, be able to inject network traffic.

However, the homomorphic fingerprinting scheme allows for any inconsistencies between data fragments to be caught before reconstruction. Any tampering will be detected because the tampered fragment's fingerprint won't match the expected encoding of the other fragments' fingerprints. In order to not be caught, an adversary would need to provide a consistent set of tampered fragments which all pass verification. This is computationally infeasible given the collision resistance of the fingerprinting scheme.

Unfortunately, given the material constraints of this project (i.e., "servers" existing on a single machine as different `localhost` ports), we are unable to fully test this.

7 Entry Points

7.1 Client API

- **Store operation**:
- **Retrieve operation**:
- **Verify operation**:

7.2 Inter-Server Communication

- **Fingerprint exchange**:
- **Fragment retrieval**:

7.3 Storage Subsystem

- **Fragment write**:
- **Fragment read**:
- **Fingerprint storage**:

8 Protected Assets

8.1 Data Blocks

8.2 Erasure-Coded Fragments

8.3 Homomorphic Fingerprints

8.4 System Resources

9 Data Flow Diagram

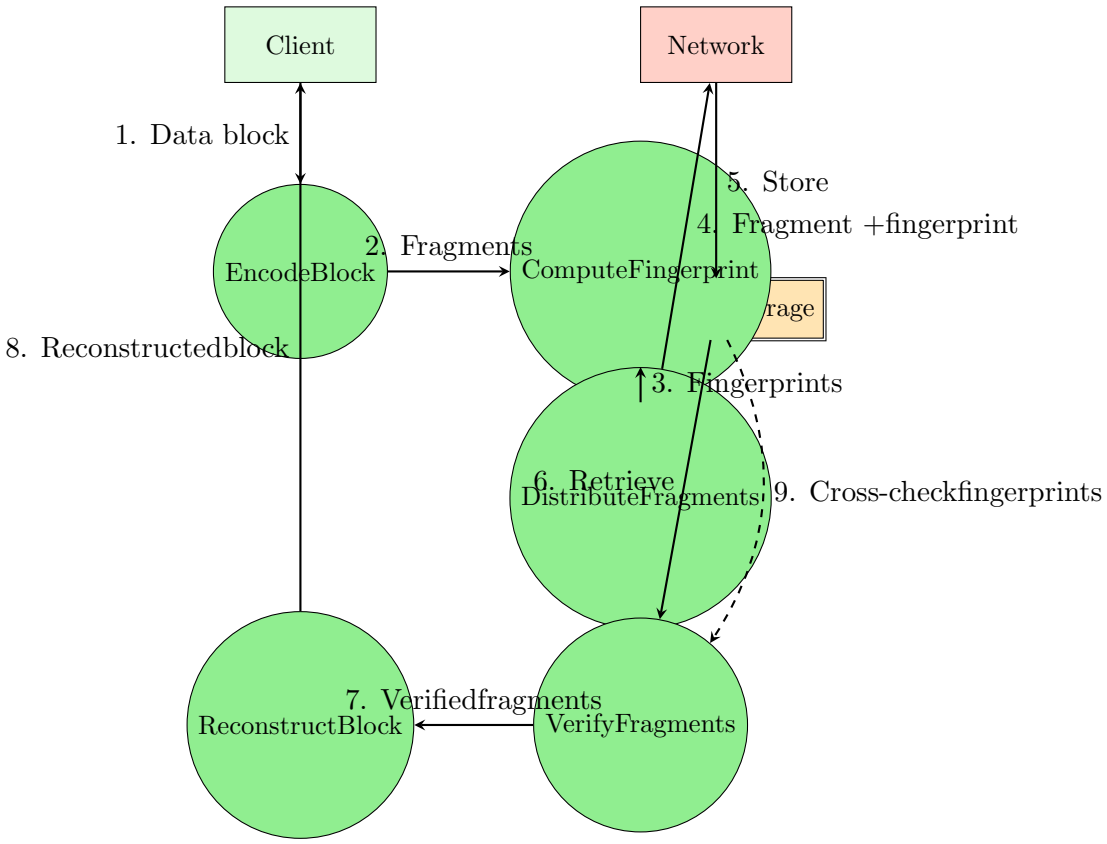

Figure 1: High-level data flow diagram for veri-store store and retrieve operations.

9.1 DFD Component Descriptions

9.1.1 Client

9.1.2 Encode Block

9.1.3 Compute Fingerprint

9.1.4 Distribute Fragments

9.1.5 Network

9.1.6 Fragment Storage

9.1.7 Verify Fragments

9.1.8 Reconstruct Block

10 Threat Analysis: STRIDE

10.1 Spoofing

10.1.1 S.1: Client Impersonation

10.1.2 S.2: Server Impersonation

10.1.3 S.3: Fingerprint Source Forgery

10.2 Tampering

10.2.1 T.1: Fragment Corruption in Storage

10.2.2 T.2: Fragment Modification in Transit

10.2.3 T.3: Fingerprint Tampering

10.2.4 T.4: Erasure Code Parameter Manipulation

10.3 Repudiation

10.3.1 R.1: Anonymous Data Storage

10.3.2 R.2: Fragment Deletion Without Logging

10.3.3 R.3: Untrackable Verification Failures

10.4 Information Disclosure

10.4.1 I.1: Fragment Eavesdropping

10.4.2 I.2: Storage Server Data Access

10.4.3 I.3: Fingerprint Analysis

10.4.4 I.4: Error Message Information Leakage

10.5 Denial of Service

10.5.1 D.1: Storage Exhaustion

11 Threat Resolutions

11.1 Spoofing Threat Resolutions

11.1.1 S.1: Client Impersonation

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.1.2 S.2: Server Impersonation

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.1.3 S.3: Fingerprint Source Forgery

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.2 Tampering Threat Resolutions

11.2.1 T.1: Fragment Corruption in Storage

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.2.2 T.2: Fragment Modification in Transit

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.2.3 T.3: Fingerprint Tampering

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.2.4 T.4: Erasure Code Parameter Manipulation

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.3 Repudiation Threat Resolutions

11.3.1 R.1: Anonymous Data Storage

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.3.2 R.2: Fragment Deletion Without Logging

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.3.3 R.3: Untrackable Verification Failures

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.4 Information Disclosure Threat Resolutions

11.4.1 I.1: Fragment Eavesdropping

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.4.2 I.2: Storage Server Data Access

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.4.3 I.3: Fingerprint Analysis

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.4.4 I.4: Error Message Information Leakage

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.5 Denial of Service Threat Resolutions

11.5.1 D.1: Storage Exhaustion

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.5.2 D.2: Fragment Unavailability

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.5.3 D.3: Verification Flooding

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.5.4 D.4: Network Bandwidth Exhaustion

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6 Elevation of Privilege Threat Resolutions

11.6.1 E.1: Code Injection via Malformed Input

- **Status**:

- **Resolution**:
- **Owner**:
- **Testing**:

11.6.2 E.2: Verification Bypass

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6.3 E.3: Cryptographic Parameter Override

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

11.6.4 E.4: Server-to-Server Privilege Escalation

- **Status**:
- **Resolution**:
- **Owner**:
- **Testing**:

12 External Security Notes

12.1 Deployment Recommendations

12.2 Known Limitations

12.3 Integration Considerations

13 Verification and Testing Plan

13.1 Security Test Suite

13.2 Code Review Process

13.3 Regression Testing

14 Document Maintenance

14.1 Review Schedule

14.2 Change Process

14.3 Version History

- **Version 0.1** (February 16, 2026): Initial draft - high-level security model with component stubs
- **Version 0.1.001** (February 16, 2026):
  - Added descriptions of external libraries
  - Added Administrator trust level and example administrators
- **Version 0.1.002** (March 1, 2026):
  - Split `.tex` file into sections for easier maintenance.
  - Changes to preamble re: formatting and spacing.
- **Version 0.1.1** (March 15, 2026):
  - Completed sections 1.1 and 1.2.
  - Inserted a `\newpage` control at the end of each section document.
  - Completed section 3
  - Completed section 6.2 & 6.4
  - Updated and verified section 4.1